

ArithSeg: Arithmetic-Sequence Factor Probing for Semiprimes

From an Alternating Arithmetic-Sequence Transform to Binomial Congruences, Lucas Structure, and the Computational Limits of Factor Localization

IamLupo

12 September 2026

Abstract

This paper records the development of the ArithSeg construction from its initial arithmetic-sequence representation to the strongest exact identities and computational observations obtained so far. The construction starts with a finite sequence of coefficients and evaluates its Newton/binomial interpolation at integer positions. A particularly structured coefficient vector,

$$[1, N-1, 1, N-1, \dots, 1, N-1],$$

turns into an alternating-sign sequence modulo either prime factor of an odd semiprime $N = pq$. This produces an exact reduction from the original arithmetic-sequence value to a binomial coefficient. For the degree parameter $d = 2z - 1$ one obtains

$$v_k(z) = -\binom{k-2}{2z-1}$$

whenever $2z - 1 < k - 1$, and hence

$$\gcd(v_k(z), N) = \gcd\left(\binom{k-2}{2z-1}, N\right).$$

When $2z - 1 < p$, the denominator of the binomial coefficient is coprime to N , reducing the factor test further to a gcd of a contiguous numerator product. Lucas' theorem then gives an exact characterization of when a prime factor divides the transformed value. This permits exact construction of factor-hitting degree parameters when the factor is known, while also exposing why factor-blind search remains difficult.

A series of experiments tests dyadic choices of k and z , binary refinement, recursive subdivision, Lucas-based construction, multiplier families, factor-blind fractional positions, random sampling, evenly spaced sampling, van der Corput sampling, density of factor-hit sets, nearby-power perturbations, and a fallback based on a prime near $\sqrt{N}$. The experiments demonstrate strong empirical recovery rates but no universal constant-size probe family. The computational study then changes the main bottleneck from exact big-integer binomial construction to linear scans of numerator intervals. Batch modular inversion and numerator-only evaluation provide large speedups, but the remaining $O(d)$ scan becomes the dominant wall at sufficiently large inputs. A direct prefix-product/binary-localization experiment is shown to be counterproductive because prefix construction costs more than the successful early-stopping scan.

The result is an exact and experimentally well-characterized factor-probing framework, not yet a proven general-purpose integer factorization algorithm. The central unresolved problem is to exploit the arithmetic structure of the hidden factor condition without paying essentially linear cost in the relevant binomial degree.

Contents

1	Purpose and scope	3
2	The initial arithmetic-sequence construction	3
3	Reduction modulo a prime divisor	4
4	The zero regime	5
5	The numerator interval representation	5
6	Lucas' theorem and exact factor-hit structure	6
6.1	Constructing a hit when the factor is known	6
7	What the first power-of-two search revealed	7
8	Dyadic misses and the absence of a universal fixed multiplier	7
9	Factor-blind positions, random sampling, and low-discrepancy sampling	8
10	Density of the hidden factor-hit set	8
11	Changing the power-of-two coordinate	9
12	The prime-near-$\sqrt{N}$ fallback	9
13	The computational bottleneck: exact binomial evaluation	9
14	Modular evaluation and batch inversion	10
15	The numerator-only evaluator	10
16	Scaling wall of the numerator scan	11
17	The prefix-product and binary-localization attempt	11
18	Exact identities collected in the ArithSeg construction	12
19	What the construction does prove	12
20	What the construction does not prove	13
21	Limitations of the search strategies	13
21.1	Dyadic k and z	13
21.2	Ordinary binary search	13
21.3	Recursive subdivision	13
21.4	Low-discrepancy and random sampling	13
21.5	Changing k locally	13
21.6	Prime-near- $\sqrt{N}$ fallback	14
21.7	Exact binomial arithmetic	14
21.8	Numerator scanning	14
21.9	Prefix-product localization	14
22	The current conceptual boundary	14

23 Potential directions	14
23.1 Block products with early gcd tests	14
23.2 Structured product trees	15
23.3 Analytic use of Lucas geometry	15
23.4 Multi- k intersection	15
23.5 Hybridization with classical close-factor methods	15
24 Conclusion	15
A Experiment record	16
B Notation summary	17

1 Purpose and scope

The purpose of this paper is to document the ArithSeg research line as it evolved, rather than to present only the final successful formulas. The central question throughout is:

Can an arithmetic-sequence construction whose coefficients depend on the composite input expose a non-trivial divisor of a semiprime through the transformed sequence values, while avoiding an exhaustive factor search?

The work is experimental in its algorithmic claims. Exact algebraic identities are stated and proved whenever possible. Empirical success rates and timings are explicitly identified as experimental observations and are not promoted to theorems.

The main object is an odd semiprime

$$N = pq, \quad p < q,$$

with p and q prime. In practice the experiments generate such N locally and evaluate arithmetic-sequence constructions at selected integer coordinates.

2 The initial arithmetic-sequence construction

Consider a finite coefficient vector

$$\mathbf{a} = (a_1, a_2, \dots, a_{d+1}).$$

The associated arithmetic-sequence/Newton interpolation is defined by

$$V_{\mathbf{a}}(k) = \sum_{j=0}^d a_{j+1} \binom{k-1}{j}, \quad (1)$$

for integer $k \geq 1$. This is the standard binomial-basis representation of the unique polynomial sequence whose forward differences at the initial point are encoded by the coefficient vector.

The crucial experimental idea was not to choose arbitrary coefficients, but to let the composite input itself appear in the coefficient sequence. The special vector is

$$\mathbf{a}_{N,z} = [1, N-1, 1, N-1, \dots, 1, N-1], \quad (2)$$

with z repetitions of the pair $(1, N-1)$. The vector has length $2z$, hence degree

$$d = 2z - 1. \quad (3)$$

We write the corresponding sequence value as

$$V_k(N, z) = \sum_{j=0}^{2z-1} c_j \binom{k-1}{j}, \quad c_j = \begin{cases} 1, & j \text{ even,} \\ N-1, & j \text{ odd.} \end{cases} \quad (4)$$

This is the fundamental transformation of the ArithSeg construction: the unknown factor is not searched for directly; rather, the composite N is inserted into the finite-difference data of a polynomial sequence.

3 Reduction modulo a prime divisor

Let r be any prime divisor of N . Since

$$N \equiv 0 \pmod{r},$$

we have

$$N - 1 \equiv -1 \pmod{r}.$$

Consequently, the coefficient pattern in (2) becomes

$$1, -1, 1, -1, \dots$$

Therefore

$$V_k(N, z) \equiv \sum_{j=0}^d (-1)^j \binom{k-1}{j} \pmod{r}. \quad (5)$$

The alternating partial-binomial identity

$$\boxed{\sum_{j=0}^d (-1)^j \binom{m}{j} = (-1)^d \binom{m-1}{d}} \quad (6)$$

holds for $0 \leq d \leq m$. Taking $m = k - 1$ and recalling that $d = 2z - 1$ is odd gives

$$\boxed{V_k(N, z) \equiv -\binom{k-2}{2z-1} \pmod{r},} \quad (7)$$

whenever $2z - 1 < k - 1$.

Since the congruence holds for every prime divisor r of N , it yields the central divisibility equivalence.

Theorem 3.1 (Binomial reduction). *Let $N = pq$ be a semiprime, let $d = 2z - 1 < k - 1$, and let $V_k(N, z)$ be defined by (4). Then*

$$\boxed{V_k(N, z) \equiv -\binom{k-2}{d} \pmod{p}, \quad V_k(N, z) \equiv -\binom{k-2}{d} \pmod{q}.} \quad (8)$$

Consequently,

$$\boxed{\gcd(V_k(N, z), N) = \gcd\left(\binom{k-2}{d}, N\right).} \quad (9)$$

Proof. Equation (7) holds modulo each prime divisor of N . Thus a prime factor of N divides $V_k(N, z)$ exactly when it divides the corresponding binomial coefficient. Taking the gcd with $N = pq$ gives (9). $\square$ $\square$

This is the decisive algebraic reduction: the apparently polynomial arithmetic-sequence problem becomes a binomial divisibility problem.

4 The zero regime

The reduction above assumes $d < k - 1$. The complementary regime is also exact. If $d \geq k - 1$, all binomial coefficients beyond $j = k - 1$ vanish and the complete alternating sum is available:

$$\sum_{j=0}^{k-1} (-1)^j \binom{k-1}{j} = (1-1)^{k-1} = 0.$$

Therefore

$$\boxed{V_k(N, z) = 0 \quad \text{when } 2z - 1 \geq k - 1.} \quad (10)$$

In this regime

$$\gcd(V_k(N, z), N) = N.$$

This creates the three principal gcd states observed throughout the experiments:

1. 1: the chosen parameter reveals neither prime factor;
2. a proper divisor p or q : the probe succeeds;
3. N : the degree has crossed into the zero regime, or the transformed value is otherwise divisible by all of N .

The third state is important because it makes the search space non-monotone in the way required for ordinary binary search: moving toward larger z can change the gcd from 1 to a factor and later to N , and the factor region need not behave like a single monotone threshold.

5 The numerator interval representation

Set

$$m = k - 2.$$

For $d < p$, the binomial coefficient

$$\binom{m}{d} = \frac{m(m-1) \cdots (m-d+1)}{d!}$$

has denominator coprime to $N = pq$, because every factor of $d!$ is strictly smaller than p .

Hence, in the principal experimental regime $d < p$,

$$\boxed{\gcd\left(\binom{m}{d}, N\right) = \gcd\left(\prod_{j=0}^{d-1} (m-j), N\right).} \quad (11)$$

Equivalently, define the interval

$$I(m, d) = [m - d + 1, m]. \quad (12)$$

Then

$$\boxed{\gcd(V_k(N, z), N) > 1 \iff I(m, d) \text{ contains a multiple of a prime factor of } N,} \quad (13)$$

provided $d < p$.

Because $d < p < q$, an interval of length d contains at most one multiple of p and cannot contain two distinct multiples of p . It also cannot contain a multiple of q more than once, and in many relevant regimes the interval lies entirely below q .

This observation changes the computational interpretation of the problem. The factor condition is no longer primarily an issue of evaluating a giant binomial coefficient. It becomes the problem of discovering an unknown multiple of an unknown divisor inside a structured integer interval.

6 Lucas' theorem and exact factor-hit structure

Let

$$m = k - 2.$$

Lucas' theorem gives, for a prime p , a base- p digit characterization of

$$\binom{m}{d} \pmod{p}.$$

Write

$$m = \sum_i m_i p^i, \quad d = \sum_i d_i p^i,$$

with $0 \leq m_i, d_i < p$. Then

$$\boxed{\binom{m}{d} \not\equiv 0 \pmod{p} \iff d_i \leq m_i \text{ for every } i.} \quad (14)$$

Consequently

$$\boxed{p \mid V_k(N, z) \iff \text{at least one base-}p \text{ digit of } d \text{ exceeds the corresponding digit of } m.} \quad (15)$$

for $d < k - 1$.

In the particularly important regime $d < p$, all non-zero base- p digits of d occur only in the units position. Lucas therefore simplifies substantially:

$$\boxed{d < p \implies p \mid \binom{m}{d} \iff d > m \bmod p.} \quad (16)$$

Thus, within $d < p$, the factor-hit set is an explicit interval condition on d modulo the hidden factor. This is one of the strongest exact structural results obtained in the ArithSeg line.

6.1 Constructing a hit when the factor is known

Let

$$r = m \bmod p.$$

The smallest integer d satisfying $d < p$ and $d > r$ is $r + 1$. If an odd degree is required, the smallest odd choice is

$$d_{\text{odd}} = \begin{cases} r + 1, & r \text{ even,} \\ r + 2, & r \text{ odd and } r + 2 < p. \end{cases} \quad (17)$$

Whenever such a d exists, the corresponding

$$z = \frac{d + 1}{2}$$

produces a p -divisible transformed value.

A more general digit construction follows directly from Lucas' theorem: choose a base- p position where $m_i + 1 < p$, set the corresponding digit of d to $m_i + 1$, and set the other digits to zero. This gives an explicit Lucas-zero candidate. The experiments used both direct constructions and odd-degree adaptations of them.

7 What the first power-of-two search revealed

A natural factor-blind strategy was to choose

$$k = 2^i$$

near $\sqrt{N}$ and to probe

$$z = 1, 2, 4, 8, \dots$$

The motivation was that powers of two provide a sparse, deterministic coordinate family and often place k between the two factors.

The resulting empirical picture was mixed. Dyadic probing frequently found a factor with only a handful of gcd evaluations, but it was not universal.

A representative progression was:

Experiment	Main observation	Result
101	Dyadic outer search	Frequent direct factor hits
102	Larger semiprime sample	558/613 direct hits (91.03%)
103	Ordinary binary refinement	0/55 bracket recoveries
104	Recursive interval subdivision	14/43 bracket recoveries
106	Lucas prediction check	6192/6192 exact matches
107	Exact Lucas construction	531/531 constructed hits; 495/531 dyadic hits

The important negative result is Experiment 103: an observed transition from 1 to N does not imply that a proper factor lies behind a monotone threshold. Consequently, ordinary binary search is not justified by the gcd state alone.

Experiment 104 showed that recursive subdivision can sometimes recover a factor window, but at the cost of many more evaluations. This established that factor-hit regions can be narrow and non-monotone.

8 Dyadic misses and the absence of a universal fixed multiplier

After the Lucas characterization was established, the next question was whether dyadic z could be repaired by testing a few nearby multipliers of powers of two, for example

$$z = c2^j$$

for odd c drawn from a small fixed set.

Experiment 108 showed a substantial empirical improvement: extending the candidate family to odd multipliers up to 31 raised the hit rate from roughly 92.66% to 99.38% on its test set. However, this was not universal.

Experiment 109 measured the exact odd multiplier required by Lucas-based construction. The required multiplier distribution was broad, reaching as high as

$$c = 2047.$$

One particularly clear case was

$$N = 40,608,793 = 4099 \cdot 9907, \quad k = 8192, \quad z = 2047,$$

showing that a fixed small collection of odd multipliers cannot provide a universal guarantee.

This is a recurring pattern in the research: a compact deterministic family can work extremely well on sampled inputs, yet Lucas' theorem exposes hidden base- p structure that prevents a constant-size factor-blind family from being justified mathematically.

9 Factor-blind positions, random sampling, and low-discrepancy sampling

A different strategy is to forget the hidden factor and sample candidate z positions directly inside the admissible interval.

Experiment 110 tested factor-blind fractional positions such as $k/2$, $k/4$, and related locations. These produced an empirical hit rate of approximately 92.95% on the tested power-of-two cases, but the exact Lucas z/k ratios ranged broadly, approximately from 0.0012 to 0.25.

Experiment 111 then tested random z sampling. Across 597 (N, k) cases the cumulative success was approximately:

Probes	Success
1	58.12%
2	72.86%
4	84.09%
8	91.96%
16	95.98%
32	98.49%
64	99.16%

Experiment 112 compared random sampling with evenly spaced sampling and van der Corput (VdC) low-discrepancy sampling. VdC was the strongest deterministic distribution in most probe budgets, reaching approximately 98.90% at 64 probes on its test population.

These experiments establish an important distinction. The factor-hit set can have substantial density, so a factor-blind sampling strategy can work surprisingly well. Yet the small number of misses shows that density alone does not imply a bounded universal probe count.

10 Density of the hidden factor-hit set

For fixed (N, k, p) define the hit set

$$\mathcal{H}_p(k) = \{z : p \mid V_k(N, z)\}. \quad (18)$$

Experiment 113 measured the empirical density of this set across 552 (N, k) cases. The average density was approximately

$$50.09\%,$$

but the range was extremely broad, from 0% to approximately 99.78%.

In particular, some cases had less than 1% density. Representative hard cases included

$$N = 11,029,649 = 1171 \cdot 9419, \quad k = 8192, \quad \text{density } 0.439\%,$$

$$N = 7,944,817 = 821 \cdot 9677, \quad k = 4096, \quad \text{density } 0.977\%,$$

$$N = 19,967,539 = 1373 \cdot 14543, \quad k = 4096, \quad \text{density } 1.17\%.$$

A special blindness case was

$$N = 75,430,919 = 8191 \cdot 9209, \quad k = 8192.$$

Here $p = k - 1$, so $m = k - 2 = p - 1$. For every $d < p$,

$$\binom{p-1}{d} \not\equiv 0 \pmod{p},$$

which means that this particular k is completely blind to the factor p throughout the clean regime.

This establishes that changing k is not merely an implementation detail: it changes the base- p digit structure of $m = k - 2$ and therefore changes which d values can expose the factor.

11 Changing the power-of-two coordinate

Experiment 115 tested nearby offsets around a power-of-two coordinate,

$$k \in \{2^i - 2, 2^i - 1, 2^i, 2^i + 1, 2^i + 2\}.$$

Among 131 power-coordinate cases, 128 were already successful at the original k , one was rescued by a nearby offset, and two remained unresolved.

The clearest rescue was

$$N = 4,180,643 = 683 \cdot 6121,$$

where the power coordinate failed but $k = 4094$ succeeded.

This suggests that nearby k values can alter the hidden base- p residue structure enough to recover a blind coordinate, but the experiment did not establish a universal finite neighborhood that always suffices.

12 The prime-near- $\sqrt{N}$ fallback

A stronger fallback emerged by setting

$$r = \text{prevprime}(\lfloor \sqrt{N} \rfloor) \tag{19}$$

and using

$$k = r.$$

Importantly, the construction does not multiply N by r ; N itself is kept unchanged.

Experiment 117 applied this fallback to 117 cases where the original dyadic search had failed. With up to 16 VdC probes, 115/117 cases succeeded, for a success rate of 98.29%. The two exceptions were

$$54,479,017 = 7369 \cdot 7393, \quad 118,026,487 = 10861 \cdot 10867.$$

In both cases $r \leq p$ because the factors are exceptionally close. These are precisely cases where classical Fermat factorization is already favorable.

Experiment 118 combined the dyadic search, the prime-near- $\sqrt{N}$ fallback, and Fermat factorization and obtained 500/500 recovery in its tested population. Experiment 119 extended the scale to primes in approximately the 10^4 – 10^5 and 10^5 – 2.5×10^5 ranges and again obtained 300/300 recovery.

These experiments demonstrate a strong practical hybrid, but they do not constitute a proof that the finite search will always recover an arbitrary semiprime efficiently.

13 The computational bottleneck: exact binomial evaluation

At first, the mathematical reduction was treated as if evaluating

$$\binom{k-2}{d}$$

were cheap. Runtime measurements showed otherwise.

Experiment 120 compared several components. For approximately 10^5 –scale factors, exact `math.comb` remained manageable. At the next scale, around

$$p \in [10^5, 10^6], \quad q \in [10^6, 10^7],$$

case times rose to several seconds, with exact binomial construction consuming essentially all measured time. GCD operations took only microseconds in comparison.

At the next scale the construction was stopped because exact binomial arithmetic had become the dominant cost. The conclusion was unambiguous:

The bottleneck is not gcd evaluation. It is constructing and manipulating the exact binomial coefficient as a huge integer.

14 Modular evaluation and batch inversion

A modular recurrence for the binomial coefficient was then tested using

$$\binom{m}{d} = \binom{m}{d-1} \frac{m-d+1}{d}.$$

A direct modular implementation avoids constructing the huge exact integer, but a naive version pays for a modular inverse at every step.

Experiment 122 found that this approach was already useful but still left substantial Python-level loop overhead. A batch-inversion implementation substantially improved the constant factor for large degrees. Representative microbenchmarks gave approximately:

d	Exact	Batch modular	Speedup
131073	0.5444 s	0.1502 s	$3.63\times$
262145	1.7215 s	0.2978 s	$5.78\times$
524289	5.4244 s	0.5895 s	$9.21\times$

All tested batch results matched the exact computation.

However, the main-regime identity (11) suggested an even simpler evaluator: there is no need to reconstruct the binomial coefficient at all.

15 The numerator-only evaluator

For $d < p$,

$$\gcd\left(\binom{m}{d}, N\right) = \gcd\left(\prod_{t=m-d+1}^m t, N\right).$$

Thus one can scan the numerator interval directly, maintaining

$$G_0 = 1, \quad G_j = \gcd(G_{j-1}(m-d+j), N). \quad (20)$$

The scan stops immediately once $G_j > 1$.

This evaluator removes both the giant exact binomial and the modular inverses. It was dramatically faster in Experiment 129. Across the tested large probes,

Method	Total time
Exact <code>math.comb</code>	16.272648 s
Numerator only	0.738909 s
Numerator + incremental gcd	0.544842 s

The numerator-plus-gcd evaluator was therefore roughly $29.9\times$ faster than exact binomial construction on that benchmark, with all tested outcomes matching.

The large probes also terminated before consuming the full interval. For example, factors were exposed after approximately 1.2×10^5 to 2.7×10^5 iterations on degrees around 2.6×10^5 – 5.2×10^5 .

16 Scaling wall of the numerator scan

The numerator transformation solves one computational bottleneck but reveals another: the scan is $O(d)$.

Experiment 130 measured the same numerator-plus-gcd method on increasing scales. For factors around 10^5 – 10^6 , five cases took about 0.57 seconds in total. For factors around 10^6 – 10^7 , five cases took about 2.14 seconds. At the next scale, with

$$p \in [10^7, 10^8], \quad q \in [10^8, 10^9],$$

the experiment had to stop after only two cases: the average case took about 9.12 seconds, with the slowest case taking about 11.12 seconds.

The slowest recorded case was approximately

$$N = 55,558,830,687,287,803,$$

with

$$p = 87,284,893, \quad q = 636,522,871, \quad k = 134,217,728,$$

and a successful probe at

$$z = 33,554,433, \quad d = 67,108,865.$$

The factor was exposed only after approximately 20.2 million numerator iterations.

This identifies the present hard wall:

current practical evaluator cost $\approx O(d)$ Python-level modular products/gcds.	(21)
---	------

The research therefore moved from the question “how do we avoid huge binomial integers?” to the harder question “how do we avoid scanning the whole structured interval?”

17 The prefix-product and binary-localization attempt

A natural idea was to precompute prefix products and then use interval products to recursively localize the hidden multiple of p . This was tested in Experiment 131.

The experiment compared an early-stopping linear numerator scan with a prefix-product construction on ten semiprimes. Every linear scan succeeded, while the prefix computation exposed a factor only after the entire prefix construction had already been performed.

The measured totals were

$$T_{\text{linear}} = 1.013207 \text{ s}, \quad T_{\text{prefix}} = 3.194658 \text{ s}.$$

Thus the prefix approach was approximately

$$3.15\times$$

slower in that experiment.

The failure is structural. Constructing the prefix array itself costs $O(d)$ modular work, so the subsequent binary localization does not remove the main work. In fact, the early-stopping linear scan often terminates substantially before the full interval is processed.

This experiment rules out the simple form of prefix-plus-binary localization as the next optimization target.

18 Exact identities collected in the ArithSeg construction

For clarity, the principal exact identities are collected here without the surrounding experimental narrative.

$$V_k(N, z) = \sum_{j=0}^{2z-1} c_j \binom{k-1}{j}, \quad c_j = \begin{cases} 1, & j \text{ even}, \\ N-1, & j \text{ odd}, \end{cases} \quad (22)$$

$$N \equiv 0 \pmod{r}, \quad r \in \{p, q\}, \quad (23)$$

$$V_k(N, z) \equiv \sum_{j=0}^d (-1)^j \binom{k-1}{j} \pmod{r}, \quad d = 2z - 1, \quad (24)$$

$$\sum_{j=0}^d (-1)^j \binom{m}{j} = (-1)^d \binom{m-1}{d}, \quad (25)$$

$$V_k(N, z) \equiv -\binom{k-2}{2z-1} \pmod{r}, \quad (26)$$

$$\gcd(V_k(N, z), N) = \gcd\left(\binom{k-2}{2z-1}, N\right), \quad (27)$$

$$V_k(N, z) = 0, \quad 2z - 1 \geq k - 1, \quad (28)$$

$$\gcd\left(\binom{m}{d}, N\right) = \gcd\left(\prod_{t=m-d+1}^m t, N\right), \quad d < p, \quad (29)$$

$$\binom{m}{d} \not\equiv 0 \pmod{p} \iff d_i \leq m_i \text{ for all base-}p \text{ digits}, \quad (30)$$

$$p \mid \binom{m}{d} \iff d > m \pmod{p}, \quad d < p, \quad (31)$$

$$p \mid V_k(N, z) \iff I(m, d) \text{ contains a multiple of } p, \quad d < p, \quad (32)$$

$$I(m, d) = [m - d + 1, m]. \quad (33)$$

Together these identities provide an exact chain

arithmetic sequence $\longrightarrow$ alternating binomial sum $\longrightarrow$ binomial divisibility $\longrightarrow$ hidden-factor interval condition

19 What the construction does prove

The algebra proves substantially more than an empirical correlation.

First, the alternating coefficient vector is not merely a heuristic device. Modulo each prime divisor, it is exactly the alternating-sign vector.

Second, the transformed value is exactly equivalent to a binomial coefficient modulo each hidden prime. The gcd relation is therefore exact.

Third, when $d < p$, the denominator of the binomial coefficient is invertible modulo N , so factor detection is exactly equivalent to detecting a multiple of a hidden factor inside a known interval.

Fourth, Lucas' theorem provides a complete digit-level description of the divisibility event.

Finally, the exact Lucas construction demonstrates that the factor-hit parameter exists in a predictable form once the factor is known.

These are mathematical results, independent of the observed success rates of any particular search strategy.

20 What the construction does not prove

The exact identities do not by themselves give a universal efficient factorization algorithm.

The most important limitation is that the useful Lucas parameter depends on the unknown prime p . The statement

$$d > m \bmod p$$

is exact, but it is not directly evaluable without already knowing p .

Similarly, the interval interpretation

$$[m - d + 1, m] \text{ contains a multiple of } p$$

is exact, but the location of that multiple is itself determined by the hidden factor.

Therefore the central algorithmic problem has not disappeared. It has been transformed into a structured search problem whose arithmetic is much clearer.

21 Limitations of the search strategies

21.1 Dyadic k and z

Powers of two give a useful sparse search family, but Experiment 102 already showed misses, and Experiment 109 demonstrated that the required odd multiplier can become large. A fixed finite set of multipliers therefore has no established universal guarantee.

21.2 Ordinary binary search

The gcd state is not monotone in z . A transition from 1 to N does not imply the existence of a factor at a unique midpoint or threshold. Experiment 103 therefore found no useful recovery from ordinary binary refinement.

21.3 Recursive subdivision

Subdivision can recover some narrow hit windows but may require many evaluations. It is a useful diagnostic strategy rather than a demonstrated asymptotically superior algorithm.

21.4 Low-discrepancy and random sampling

These methods exploit the empirical density of hit sets. They can achieve high success with surprisingly few probes, but the observed hard cases show that neither randomness nor low discrepancy gives a deterministic universal bound.

21.5 Changing k locally

Nearby values of k can rescue some blind coordinates, but the experiments do not establish a fixed neighborhood size that always succeeds.

21.6 Prime-near- $\sqrt{N}$ fallback

The fallback works very well empirically and integrates naturally with Fermat for unusually close factors. Nevertheless, it remains an empirical hybrid rather than a proof that every semiprime is recovered within a bounded probe budget.

21.7 Exact binomial arithmetic

Constructing `math.comb` values becomes the dominant cost well before the algebra itself becomes interesting at large scale.

21.8 Numerator scanning

The numerator-only evaluator removes the giant-integer bottleneck, but it has linear dependence on d . This becomes expensive when the successful factor location lies tens of millions of iterations into the interval.

21.9 Prefix-product localization

Experiment 131 showed that a straightforward prefix construction does not fix the linear-scan problem. It pays the full linear cost before the proposed binary localization can help.

22 The current conceptual boundary

The research has therefore reached a fairly clean boundary.

On the mathematical side, the original arithmetic-sequence object has been reduced to a binomial coefficient, then to Lucas digit conditions, and finally to a hidden-multiple interval problem.

On the computational side, the evaluator has been reduced from giant exact binomial arithmetic to a simple modular numerator scan, yielding large speedups.

The unresolved step is now sharply stated:

Find a way to determine whether a structured interval contains a multiple of an unknown prime divisor of N , or to localize such a multiple, without performing essentially one modular operation per candidate integer in the interval.

A successful method would have to exploit additional structure in k , d , the power-of-two coordinate, Lucas digit geometry, or products over structured subintervals. Merely changing the data structure while retaining an $O(d)$ preprocessing pass is unlikely to change the fundamental scaling.

23 Potential directions

Several directions follow naturally from the exact structure, without claiming that any has yet succeeded.

23.1 Block products with early gcd tests

Partition the numerator interval into blocks and test gcds of block products. This remains $O(d)$ if every element is explicitly multiplied, but it can reveal whether a practical cache- and gcd-level improvement exists before attempting more sophisticated product structures.

23.2 Structured product trees

A genuinely useful localization method would need a product representation that supports interval products without first enumerating every factor. This suggests product trees or factorial-ratio representations, but any such approach must be benchmarked carefully because the cost of constructing the tree can simply reproduce the original linear work.

23.3 Analytic use of Lucas geometry

The exact condition $d > m \bmod p$ suggests looking for search coordinates k for which the map

$$p \mapsto (k - 2) \bmod p$$

has unusually strong constraints when $p \mid N$. The challenge is to obtain such constraints without enumerating candidate divisors.

23.4 Multi- k intersection

Because changing k changes the Lucas digit structure, one could combine several factor-blind k values and seek a candidate factor that consistently induces a hit pattern. Any such method should be evaluated against the baseline probability expected from ordinary hit-set density.

23.5 Hybridization with classical close-factor methods

The prime-near- $\sqrt{N}$ fallback already suggests a practical hybrid: use ArithSeg where the factor-hit density is favorable, and use Fermat when the factors are close enough that the fallback coordinate is below the smaller factor. The important future question is whether the handoff boundary can be detected from N alone and whether the combined procedure has useful asymptotic behavior.

24 Conclusion

The ArithSeg construction began as an arithmetic-sequence experiment in which the composite input itself controlled the finite-difference coefficients. The decisive transformation was the repeated coefficient pattern

$$[1, N - 1, 1, N - 1, \dots],$$

which becomes alternating signs modulo every hidden prime factor.

That transformation produced the exact identity

$$V_k(N, z) \equiv -\binom{k-2}{2z-1} \pmod{p},$$

and consequently

$$\gcd(V_k(N, z), N) = \gcd\left(\binom{k-2}{2z-1}, N\right).$$

In the regime $2z - 1 < p$, the binomial coefficient can be replaced by a gcd of a contiguous numerator product, and Lucas' theorem reduces divisibility to the exact inequality

$$2z - 1 > (k - 2) \bmod p.$$

The algebra is therefore strong: the transformed arithmetic sequence is not merely correlated with factorization. It is exactly equivalent to a family of binomial divisibility events.

The algorithmic limitation is equally clear. The factor-dependent inequality contains the unknown prime, and factor-blind search can miss the useful degree. Random and low-discrepancy probing obtain high empirical success rates, while power-of-two coordinates and a prime-near- $\sqrt{N}$ fallback provide useful practical structure. Nevertheless, no universal finite probe family has been established.

The computational work followed the same path. Exact binomial construction was replaced by modular recurrence, then by batch inversion, and finally by the stronger numerator-only gcd evaluator. This produced large speedups, but the ultimate cost became an $O(d)$ scan. A simple prefix/binary localization experiment confirmed that naive preprocessing merely pays that same linear cost in advance.

The present state of ArithSeg is therefore best described as an exact factor-probing framework with a well-understood algebraic core and a sharply identified computational bottleneck. The remaining problem is no longer to discover another equivalent identity, but to exploit the identities already found in a way that bypasses the linear hidden-multiple search.

A Experiment record

The following list records the principal experiments relevant to the final state of the construction.

Experiment	Theme	Principal result
101	Dyadic probing	Frequent direct hits with sparse power-of-two probes.
102	Larger dyadic search	558/613 direct hits; misses remained.
103	Binary refinement	0/55 recovery after $1 \rightarrow N$ brackets.
104	Recursive subdivision	14/43 bracket recoveries; factor windows can be narrow/non-monotone.
106	Lucas verification	6192/6192 exact Lucas/gcd matches.
107	Lucas construction	531/531 constructed factor hits; 495/531 dyadic hits.
108	Odd multipliers	Hit rate improved to about 99.38% on the tested sample, but not universally.
109	Exact multiplier distribution	Required odd multipliers reached 2047.
110	Factor-blind fractions	About 92.95% hit rate; exact z/k ratios broad.
111	Random probing	About 99.16% by 64 probes on 597 cases.
112	Sampling comparison	VdC was generally strongest at moderate/high probe budgets.
113	Hit-set density	Mean about 50.09%, but density ranged from 0% to about 99.78%.
115	Nearby k values	Nearby offsets rescued one of three displayed failures; not universal.
117	Prime-near- $\sqrt{N}$ fallback	115/117 fallback attempts recovered the factor.
118	Hybrid	500/500 recovery in the tested population.
119	Larger hybrid scale	300/300 recovery in the tested population.
120	Runtime scaling	Exact <code>math.comb</code> became the dominant bottleneck.
122	Modular recurrence	Correct but slowed by repeated modular inverses.
126	Batch inversion	Up to about $9.21 \times$ faster than exact binomial in microbenchmarks.

128	Batch vs exact	Batch modular computation about $5.92\times$ faster in tested probes.
129	Numerator-only gcd	About $29.9\times$ faster than exact binomial in the tested large probes.
130	Large-scale numerator scan	Linear numerator scanning became the new hard wall.
131	Prefix/binary localization	Prefix approach about $3.15\times$ slower than early-stopping linear scan.

B Notation summary

Symbol	Meaning
N	Odd semiprime pq
p, q	Prime factors, with $p < q$
k	Arithmetic-sequence evaluation index
z	Repetition parameter in $[1, N - 1]$ pairs
d	Degree, $d = 2z - 1$
$V_k(N, z)$	Arithmetic-sequence value
m	$k - 2$ in the binomial reduction
$I(m, d)$	Numerator interval $[m - d + 1, m]$
$H_p(k)$	Set of z values whose transformed value is divisible by p